GOOGLE | 1POINT3ACRES, PRACHUB | TOPIC 8 OF 42 | CORE 90 CHAPTER TOPIC

Core 90 - Dijkstra and shortest paths

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4
MEDIUM

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Core 90 - Dijkstra and shortest paths | 4 questions | tap a title to jump to it

M	1. Train / Flight Schedule Reachability	1Point3Acres	2026-10-08
M	2. Shortest Path Nodes and Escaping a Pursuer	1Point3Acres	2026-09-29
M	3. Find A Threshold-Limited Path With Minimum Required Safety	PracHub	2026-07-06
M	4. Minimum Direction Violations in a Directed Graph	1Point3Acres	2026-07-19

STUDY PATH: question 49 of 167 (Core 90)

Priority score 6.0 | Dijkstra and shortest paths | asked 12 days ago | frequency Low

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 1 of 4

Train / Flight Schedule Reachability

ASKED 2026-10-08 | 12 DAYS AGO | LAST 30 DAYS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 182/228**

ABOUT THIS QUESTION

Topic: Core 90 - Dijkstra and shortest paths

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 2092 Find All People With Secret (related)

Frequency: Low | Last asked: 2026-10-08 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/train-schedule-reachability

QUESTION

Given trains (or flights) as (from, to, departure, arrival), decide whether you can travel from a source to a destination, boarding a later leg only if it departs after you arrive. Graph reachability with time state, solved with BFS / DFS.

Input: a schedule of legs, each with a departure station, an arrival station, a departure time, and an arrival time; plus a source and a destination.
Output: whether the destination is reachable from the source, where a leg can only be taken if it departs after you arrive at its departure station.
The same prompt appears as a flight schedule between airports in new-grad R1.

EXAMPLES

```
trains = [
  ("A", "Y", dep=1, arr=20),
  ("A", "Y", dep=2, arr=5),
  ("Y", "B", dep=6, arr=9)
]
source = "A", destination = "B"

Output: true — the first A->Y train arrives too late
    for the Y->B connection, but the second one makes
    it.
```

APPROACH

Implement BFS / DFS over (station, arrival time) states that only follows legs departing after the current arrival.
Hand-trace the example, including the late A->Y leg that must be rejected.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether a connection with departure equal to arrival is allowed, whether a leg can arrive at the same time it leaves, whether times are integers, and whether the answer is yes or no or the route.

INTERVIEW-DAY PLAN

BRUTE FORCE TO SAY OUT LOUD

Search over (station, arrival time) states following any leg that departs later. O(legs^2).

OPTIMAL IDEA

Sort the legs by departure time and keep earliest[station]. The source starts at minus infinity. For each leg, if earliest[origin] is before (or equal to, if allowed) the departure, set earliest[destination] = min(earliest[destination], arrival). The destination is reachable when it has an entry. O(L log L); the table also gives the earliest arrival time, and a parent leg per station gives the route.

DRY-RUN THESE CASES

The example with the late first A to Y leg, source equals destination, no legs, a connection with arrival exactly equal to departure, parallel legs, a cycle in the stations.

LIKELY FOLLOW-UPS

Return the route or the earliest arrival, many queries on one schedule (group by origin and use a time-ordered heap), or minimum number of legs.

PITFALLS

Ignoring time and treating it as station reachability (the stated trap). Mixing up strict and non-strict connection times. Processing legs in input order instead of departure order.

HOW TO THINK ABOUT IT

Plain station-to-station reachability is wrong because a leg is only usable if you are already at its departure station before it leaves. Time is part of the state, so ask what you need to know about each station: the earliest time you can be there. Process the legs in order of departure time; a leg is usable if the earliest arrival at its origin is not later than its departure, and then it can improve the earliest arrival at its destination. One sort and one pass answer reachability, and the same table gives the earliest arrival time.

Basis: LeetCode 2092, Find All People With Secret, and LeetCode 787, Cheapest Flights Within K Stops: reachability where time is part of the state.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def reachable(legs, src, dst):
    stack, seen = [(src, -INF)], set()
    while stack:
        station, now = stack.pop()
        if station == dst:
            return True
        if (station, now) in seen:
            continue
        seen.add((station, now))
        for a, b, dep, arr in legs:
            if a == station and dep > now:
                stack.append((b, arr))
    return False
```

BRUTE FORCE

TIME	SPACE
O(legs^2)	O(legs)

WHY IT IS SLOW

Every state rescans all legs and the same station is reached at many times, O(legs^2). Sorting the legs by departure and keeping only the earliest arrival per station answers in O(L log L).

OPTIMAL SOLUTION · Sort by departure and keep earliest arrivals

TIME	SPACE
O(L log L)	O(S)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the earliest arrival time at the destination.

The table already holds it: earliest_arrivals(legs, source)[dest], or infinity when the station is unreachable. That is the same one pass, so it costs nothing extra.

FOLLOW-UP 2 Print the route, not just yes or no.

Remember the leg that last improved each station's earliest arrival, then walk back from the destination to the source and reverse. Because only a strictly better arrival replaces the stored leg, the stored legs form a valid chain in time.

FOLLOW-UP 3 Many queries on the same schedule: what changes in the graph representation?

Group the legs by origin station and sort each list by departure time. For a query the earliest arrival is relaxed with a heap ordered by time (Dijkstra on time), and the first leg you may board at a station is found with bisect on its departure list, so each query costs O(L log S) instead of re-sorting every time. If the source is fixed and the queries only differ by destination, keep the earliest-arrival table and answer each in O(1).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Train / Flight Schedule Reachability
#
# Given trains (or flights) as (from, to, departure,
# arrival), decide whether you can travel from a source to
# a destination, boarding a later leg only if it departs
# after you arrive. Graph reachability with time state,
# solved with BFS / DFS. Input: a schedule of legs, each
# with a departure station, an arrival station, a
# departure time, and an arrival time; plus a source and a
# destination. Output: whether the destination is
# reachable from the source, where a leg can only be taken
# if it departs after you arrive at its departure station.
# The same prompt appears as a flight schedule between
# airports in new-grad R1. Examples: trains = [ ("A", "Y",
# dep=1, arr=20), ("A", "Y", dep=2, arr=5), ("Y", "B",
# dep=6, arr=9) ] source = "A", destination = "B" Output:
# true — the first A->Y train arrives too late for the Y->
B
# connection, but the second one makes it.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the earliest arrival time at the destination.
# 2. Print the route, not just yes or no.
# 3. Many queries on the same schedule: what changes in
# the graph representation?
#
# INTERVIEW NOTES
# Plain station-to-station reachability is wrong because a
# leg is only usable if you are already at its departure
# station before it leaves. Time is part of the state, so
# ask what you need to know about each station: the
# earliest time you can be there. Process the legs in
# order of departure time; a leg is usable if the earliest
# arrival at its origin is not later than its departure,
# and then it can improve the earliest arrival at its
# destination. One sort and one pass answer reachability,
# and the same table gives the earliest arrival time.
#
# BRUTE FORCE (say it first): Explore every (station,
# arrival time) state
# Search from the source over states (station, time you
# arrived), following every leg that departs after the
# current arrival. Correct and the obvious first answer,
# but each state rescans all legs. Time O(legs^2). Space
# O(legs).
```

```
# HELPERS for the brute force
INF = float("inf")


# BRUTE FORCE (say it first, it also checks the answers)
def reachable_brute(legs, source, dest, strict=True):
    # the oracle: search over (station, arrival time)
    # states
    seen, stack = set(), [(source, -INF)]
    while stack:
        station, now = stack.pop()
        if station == dest:
            return True
        if (station, now) in seen:
            continue
        seen.add((station, now))
        for frm, to, dep, arr in legs:
            ok = dep > now if strict else dep >= now
            if frm == station and ok:
                stack.append((to, arr))
    return False


# BASE QUESTION (optimal): Sort by departure and keep
# earliest arrivals
# Idea: Sort the legs by departure time. Keep
# earliest[station] = earliest known arrival, with the
# source at minus infinity.
def earliest_arrivals(legs, source, strict=True):
    """Time is part of the state: keep the earliest time
    we can be at each station. Legs go in departure
    order, so every leg that could feed this one is
    already done."""
```

```
    earliest = {source: -INF}
    by_departure = sorted(legs, key=lambda x: x[2])
    for frm, to, dep, arr in by_departure:
        have = earliest.get(frm, INF)
        # we must already be there before it leaves
        if have < dep or (not strict and have == dep):
            if arr < earliest.get(to, INF):
                earliest[to] = arr
    return earliest
```

```
# BASE QUESTION (optimal): Sort by departure and keep
# earliest arrivals
# Idea: Sort the legs by departure time. Keep
# earliest[station] = earliest known arrival, with the
# source at minus infinity.
def reachable(legs, source, dest, strict=True):
    return dest in earliest_arrivals(legs, source, strict)


# FOLLOW-UP 2: Print the route, not just yes or no.
# Answer: Remember the leg that last improved each
# station's earliest arrival, then walk back from the
# destination to the source and reverse.
def itinerary(legs, source, dest, strict=True):
    """Follow-up: the route. Remember the leg that last
    improved each station, then walk back from the end."""
    earliest, came = {source: -INF}, {}
    for leg in sorted(legs, key=lambda x: x[2]):
        frm, to, dep, arr = leg
        have = earliest.get(frm, INF)
        usable = have < dep or (not strict
                                and have == dep)
        if usable and arr < earliest.get(to, INF):
            earliest[to], came[to] = arr, leg
    if dest not in earliest:
        return None
    route = []
    while dest != source:
        leg = came[dest]
        route.append(leg)
        dest = leg[0]
    return route[::-1]


if __name__ == "__main__":
    legs = [("A", "Y", 1, 20), ("A", "Y", 2, 5),
            ("Y", "B", 6, 9)]
    assert reachable(legs, "A", "B")
    # without the early A to Y train the connection is
    # lost
    assert not reachable(legs[:1] + legs[2:], "A", "B")
    assert earliest_arrivals(legs, "A")["B"] == 9
    assert itinerary(legs, "A", "B") == [legs[1], legs[2]]
    assert itinerary(legs, "B", "A") is None
    assert reachable(legs, "A", "A")
```

```
    touch = [("A", "B", 1, 5), ("B", "C", 5, 8)]
    assert not reachable(touch, "A", "C")
    assert reachable(touch, "A", "C", strict=False)
    import random
    for _ in range(300):
        many = []
        for _ in range(random.randint(0, 8)):
            dep = random.randint(0, 9)
            many.append((random.choice("ABCD"),
                         random.choice("ABCD"), dep,
                         dep + random.randint(1, 4)))
        src = random.choice("ABCD")
        dst = random.choice("ABCD")
        for strict in (True, False):
            want = reachable_brute(many, src, dst, strict)
            got = reachable(many, src, dst, strict)
```

```
            assert got == want
```

NOTES FROM CANDIDATES

Pitfalls: Plain station-level reachability is wrong: in the example, A->Y->B exists as a path, but only one of the two A->Y legs makes the connection.
Clarify whether a connection with departure exactly equal to arrival is allowed.

Follow-ups: Follow-ups push on graph representation

PREPARATION

Implement BFS / DFS over (station, arrival time) states that only follows legs departing after the current arrival.
Hand-trace the example, including the late A->Y leg that must be rejected.

STUDY PATH: question 50 of 167 (Core 90)

Priority score 5.5 | Dijkstra and shortest paths | asked 21 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 2 of 4

Shortest Path Nodes and Escaping a Pursuer

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 212/228

ABOUT THIS QUESTION

Topic: Core 90 - Dijkstra and shortest paths

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/shortest-path-nodes-and-pursuer-escape

QUESTION

Unweighted undirected graph: shortest path from Alice's node to a target (BFS). Follow-up 1: all nodes that lie on at least one shortest path. Follow-up 2: Bob starts at another node at the same time; decide whether Alice can follow some shortest path to the target without being caught.

Input: an undirected, unweighted graph; Alice starts at one node and must reach a target node.
Return the shortest path (length) from Alice to the target.
Follow-up 1: find all nodes that can appear on some shortest path from Alice to the target.
Follow-up 2: Bob is at another node. Alice picks one shortest path and starts moving; Bob starts moving at the same time. Decide whether Alice can reach the target safely without being caught by Bob.

APPROACH

Run BFS from both the start and the target; a node v is on some shortest path exactly when distA[v] + distT[v] == distA[target]. For the pursuer follow-up, add a BFS from Bob and compare arrival times node by node along candidate shortest paths; practice turning "exists a safe shortest path" into a search over only those on-path nodes where Alice arrives strictly before Bob.
Test with disconnected targets, Bob starting on the target, and multiple equally short routes where only some are safe.

HOW TO THINK ABOUT IT

BFS from Alice gives dA and BFS from the target gives dT. A node is on some shortest path exactly when dA[v] + dT[v] equals the shortest length. For the pursuer, BFS from Bob gives dB. State the rule you assume: Alice is caught at a node v if Bob can be there no later than she is (dB[v] <= dA[v]). Alice is safe when a shortest path exists whose every node has dB[v] > dA[v].

HOW TO THINK ABOUT IT

Basis: Written for this question (BFS layers, as in LeetCode 1976, Number of Ways to Arrive at Destination).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(graph, alice, bob, target):
    dA = bfs_dist(graph, alice)
    dB = bfs_dist(graph, bob)
    paths = all_paths(graph, alice, target)
    for path in paths:
        if len(path) - 1 != dA[target]:
            continue  # not a shortest path
        if all(dB[v] > dA[v] for v in path):
            return True  # Bob is always late
    return False
```

TIME	SPACE
exponential in the worst ...	O(paths)
exponential in the worst case	

WHY IT IS SLOW

The number of shortest paths can be exponential. Three BFS runs (from Alice, the target and Bob) decide every node in O(V + E).

OPTIMAL SOLUTION · Three BFS runs and a safe shortest-path DAG

TIME	SPACE
O(V + E)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if Bob can also meet Alice in the middle of an edge?

Then arriving at the same node at the same time is not the only way to be caught: Alice moving u -> v while Bob moves v -> u swaps places. Add that case by also rejecting an edge (u, v) when dB[v] == dA[u] and dB[u] == dA[v]. Always clarify what 'caught' means first.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Shortest Path Nodes and Escaping a
# Pursuer
#
# Unweighted undirected graph: shortest path from Alice's
# node to a target (BFS). Follow-up 1: all nodes that lie
# on at least one shortest path. Follow-up 2: Bob starts
# at another node at the same time; decide whether Alice
```

```
# can follow some shortest path to the target without
# being caught. Input: an undirected, unweighted graph;
# Alice starts at one node and must reach a target node.
# Return the shortest path (length) from Alice to the
# target. Follow-up 1: find all nodes that can appear on
# some shortest path from Alice to the target. Follow-up
# 2: Bob is at another node. Alice picks one shortest path
# and starts moving; Bob starts moving at the same time.
# Decide whether Alice can reach the target safely without
# being caught by Bob.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if Bob can also meet Alice in the middle of an
# edge?
#
# INTERVIEW NOTES
# BFS from Alice gives dA and BFS from the target gives
# dT. A node is on some shortest path exactly when dA[v] +
# dT[v] equals the shortest length. For the pursuer, BFS
# from Bob gives dB. State the rule you assume: Alice is
# caught at a node v if Bob can be there no later than she
# is (dB[v] <= dA[v]). Alice is safe when a shortest path
# exists whose every node has dB[v] > dA[v].
#
# BRUTE FORCE (say it first): Enumerate every shortest
# path
# Depth-first search all simple paths of the shortest
# length from Alice to the target and check each node
# against Bob's arrival time. Correct and the oracle; the
# number of shortest paths can be huge. Time exponential
# in the worst case. Space O(paths).


from collections import deque


# HELPERS for the brute force
def bfs(adj, s):
```

```
    # distance from s to every reachable node
    d = {s: 0}
    q = deque([s])
    while q:
        u = q.popleft()
        for v in adj[u]:
            if v not in d:
                d[v] = d[u] + 1
                q.append(v)
    return d


# BRUTE FORCE (say it first): Enumerate every shortest
# path | time exponential in the worst case
def escape_brute(adj, a, t, b):
    def bfs(s):
        d = {s: 0}
        q = deque([s])
        while q:
            u = q.popleft()
            for v in adj[u]:
                if v not in d:
                    d[v] = d[u] + 1
                    q.append(v)
        return d

    da, db = bfs(a), bfs(b)
    if t not in da:
        return None, set(), False
    nodes, safe = set(), False

    def go(u, path):
        nonlocal safe
        if len(path) - 1 > da[t]:
            return
        if u == t:
            if len(path) - 1 == da[t]:
                nodes.update(path)
                if all(
                    db.get(v, 10**9) > i
                    for i, v in enumerate(path)
                ):
                    safe = True
            return
        for v in adj[u]:
```

```
            if v not in path:
                go(v, path + [v])

    go(a, [a])
    return da[t], nodes, safe


# HELPERS for the base question
FAR = 10 ** 9


# BASE QUESTION (optimal): Three BFS runs and a safe
# shortest-path DAG
# Idea: dA, dT and dB come from BFS. The nodes on some
# shortest path are those with dA[v] + dT[v] == dA[t].
def shortest_length(adj, a, t):
    # the base question: edges on the shortest path
    return bfs(adj, a).get(t)


# QUESTION FOLLOW-UP 1: all nodes on a shortest path
def nodes_on_shortest_paths(adj, a, t):
    da, dt = bfs(adj, a), bfs(adj, t)
    if t not in da:
        return set()
    # a node is on some shortest path if the two
    # distances add up to the full length
    return {v for v in da if v in dt
            and da[v] + dt[v] == da[t]}


# QUESTION FOLLOW-UP 2: can Alice escape Bob?
def can_escape(adj, a, t, b, meet_on_edge=False):
    da, db = bfs(adj, a), bfs(adj, b)
    if t not in da:
        return False
    on_path = nodes_on_shortest_paths(adj, a, t)

    def safe(u, v):
        # Bob must reach v strictly after Alice does
        if db.get(v, FAR) <= da[v]:
            return False
        # FOLLOW-UP 1: meet_on_edge=True: they meet inside
        # an edge
        # optional: they swap places on the edge u-v
```

```
        swap = db.get(v, FAR) == da[u]
        swap = swap and db.get(u, FAR) == da[v]
        return not (meet_on_edge and swap)

    # Bob standing on Alice's start catches her at once
    if db.get(a, FAR) <= 0:
        return False
    # walk the shortest-path layers using safe nodes only
    frontier, seen = [a], {a}
    while frontier:
        nxt = []
        for u in frontier:
            for v in adj[u]:
                if v in on_path and da[v] == da[u] + 1 \
                        and v not in seen and safe(u, v):
```

```
                    seen.add(v)
                    nxt.append(v)
        frontier = nxt
    return t in seen


if __name__ == "__main__":
    # a square 0-1-3 and 0-2-3 plus a tail 3-4
    adj = {0: [1, 2], 1: [0, 3], 2: [0, 3],
           3: [1, 2, 4], 4: [3]}
    assert shortest_length(adj, 0, 4) == 3
    on_path = nodes_on_shortest_paths(adj, 0, 4)
    assert on_path == {0, 1, 2, 3, 4}
    # Bob waiting at the target catches Alice there
    assert not can_escape(adj, 0, 4, 4)
    chain = {0: [1], 1: [0, 2], 2: [1, 3], 3: [2]}
    # Bob starts at the target and reaches it first
    assert not can_escape(chain, 0, 3, 3)
    # Bob starts behind Alice: she stays ahead
    assert can_escape(chain, 1, 3, 0)
```

NOTES FROM CANDIDATES

Asked as an L4 onsite coding round.
Follow-up 2 has an open definition of "caught": clarify whether it means arriving at the same node at the same time, Bob arriving first, or also meeting on an edge; also clarify whether Alice may choose her shortest path knowing Bob's position.

PREPARATION

Run BFS from both the start and the target; a node v is on some shortest path exactly when distA[v] + distT[v] == distA[target].
For the pursuer follow-up, add a BFS from Bob and compare arrival times node by node along candidate shortest paths; practice turning "exists a safe shortest path" into a search over only those on-path nodes where Alice arrives strictly before Bob.
Test with disconnected targets, Bob starting on the target, and multiple equally short routes where only some are safe.

STUDY PATH: question 51 of 167 (Core 90)

Priority score 3.02 | Dijkstra and shortest paths | asked 106 days ago

MEDIUM | IN APP LC 1102 | PracHub | Technical Screen | 3 of 4

Find A Threshold-Limited Path With Minimum Required Safety

POSTED 2026-07-06 | 106 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 136/228

ABOUT THIS QUESTION

Topic: Core 90 - Dijkstra and shortest paths

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1102 Path With Maximum Minimum Value

LeetCode match: LC 1102 Path With Maximum Minimum Value (inferred from overview)

Posted on PracHub: 2026-07-06 (updated 2026-07-08)

Original: https://prachub.com/coding-questions/find-a-threshold-limited-path-with-minimum-required-safety

QUESTION

Overview: Practice a graph path interview problem that asks for the minimum required safety value along a threshold-limited path. The prompt targets minimax path reasoning, filtered edges, Dijkstra-style relaxation, disconnected graphs, and edge-case handling.

APPROACH

Minimax Dijkstra: heap keyed by the best bottleneck (min safety along the path) while skipping edges beyond the threshold; or binary search the safety value + BFS.

HOW TO THINK ABOUT IT

When the quality of a path is decided by its weakest edge, the best path does not need to be short, only strong. Two equivalent tools: add edges from strongest to weakest with Union-Find until source and target connect (that edge's value is the answer), or run Dijkstra with a max-heap where extending a path takes the minimum so far. Disconnected graphs must return the 'no path' value.

Basis: LeetCode 1102 (Path With Maximum Minimum Value) and the bottleneck path on a graph. Assumed form (confirm it): edges (u, v, safety); the safety of a path is its smallest edge; find the best achievable path safety from source to target, or -1 if disconnected.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(graph, src, dst):
    best = -INF
    for path in all_paths(graph, src, dst):
        weakest = min(edge_safety(path))
        best = max(best, weakest)
    return best
```

BRUTE FORCE

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

Exponential. Adding edges from strongest to weakest in a Union-Find until the source and target connect is O(E log E).

OPTIMAL SOLUTION · Union-Find adding edges from strongest to weakest

TIME	SPACE
O(E log E)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Use a max-heap search instead of Union-Find.

Dijkstra where the value of a path is the minimum edge so far and you maximise it: pop the best bottleneck, extend with min(value, edge). O(E log V); Union-Find is simpler when you need only the value.

FOLLOW-UP 2 Answer many (source, target) queries.

Build the maximum spanning tree once (Kruskal) and answer each query as the minimum edge on the tree path, with binary lifting for O(log V) per query.

FOLLOW-UP 3 Edges below a threshold are forbidden (threshold-limited).

Filter them out before the search (or stop adding edges below the threshold in the Union-Find loop); if source and target are still disconnected the answer is -1.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find A Threshold-Limited Path With
# Minimum Required Safety
#
# Practice a graph path interview problem that asks for
# the minimum required safety value along a threshold-
# limited path. The prompt targets minimax path reasoning,
# filtered edges, Dijkstra-style relaxation, disconnected
# graphs, and edge-case handling.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Use a max-heap search instead of Union-Find.
# 2. Answer many (source, target) queries.
# 3. Edges below a threshold are forbidden (threshold-
# limited).
#
# INTERVIEW NOTES
```

```
# When the quality of a path is decided by its weakest
# edge, the best path does not need to be short, only
# strong. Two equivalent tools: add edges from strongest
# to weakest with Union-Find until source and target
# connect (that edge's value is the answer), or run
# Dijkstra with a max-heap where extending a path takes
# the minimum so far. Disconnected graphs must return the
# 'no path' value.
#
# BRUTE FORCE (say it first): Try every simple path
# Depth-first search over simple paths from the source,
# tracking the minimum edge safety, and keep the best at
# the target. Exact and the oracle; exponential. Time
# exponential. Space O(path).


from collections import defaultdict
import heapq


# BRUTE FORCE (say it first): Try every simple path | time
# exponential
def bottleneck_brute(n, edges, s, t):
    adj = defaultdict(list)
    for u, v, w in edges:
        adj[u].append((v, w))
        adj[v].append((u, w))
    best = [-1]
```

```
    def go(u, seen, worst):
        if u == t:
            best[0] = max(best[0], worst)
            return
        for v, w in adj[u]:
            if v not in seen:
                go(v, seen | {v}, min(worst, w))

    go(s, {s}, float("inf"))
    return (
        best[0]
        if best[0] != float("inf")
        else float("inf")
    )


# HELPERS for the base question
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


# BASE QUESTION (optimal): Union-Find adding edges from
# strongest to weakest
# Idea: Sort the edges by safety, strongest first, and
# union their endpoints one by one.
def best_path_safety(n, edges, s, t, threshold=None):
    # edges are (u, v, safety). The safety of a path is
    # its
    # weakest edge; return the best safety, or -1 if s and
    # t
    # are not connected through edges >= threshold
    if s == t:
        return float("inf")
    uf = UnionFind(n)
```

```
    # add the safest edges first: the edge that joins s
```

```
    # and t
    # is the weakest edge of the best path
    for u, v, w in sorted(edges, key=lambda e: -e[2]):
        if threshold is not None and w < threshold:
            break
        uf.union(u, v)
        if uf.find(s) == uf.find(t):
            return w
    return -1


# FOLLOW-UP 1: Use a max-heap search instead of Union-
# Find.
# Answer: Dijkstra where the value of a path is the
# minimum edge so far and you maximise it: pop the best
# bottleneck, extend with min(value, edge). O(E log V);
# Union-Find is simpler when you need only the value.
def best_path_safety_heap(n, edges, s, t, threshold=None):
    # same answer with a max-heap (Dijkstra on the
    # bottleneck)
    graph = [[] for _ in range(n)]
    for u, v, w in edges:
        if threshold is None or w >= threshold:
            graph[u].append((v, w))
            graph[v].append((u, w))
    best = [-1] * n
    best[s] = float("inf")
    heap = [(-best[s], s)]
    while heap:
        neg, u = heapq.heappop(heap)
        if -neg < best[u]:
            continue
        for v, w in graph[u]:
            # extending a path can only lower its weakest
            # edge
            cand = min(-neg, w)
            if cand > best[v]:
                best[v] = cand
                heapq.heappush(heap, (-cand, v))
    return best[t]


if __name__ == "__main__":
    edges = [(0, 1, 5), (1, 2, 3), (0, 2, 1), (2, 3, 4)]
    # 0-1-2-3 has weakest edge 3; 0-2-3 only 1
```

```
    assert best_path_safety(4, edges, 0, 3) == 3
    assert best_path_safety_heap(4, edges, 0, 3) == 3
    # edges below the threshold are forbidden
    assert best_path_safety(4, edges, 0, 3, 4) == -1
    assert best_path_safety_heap(4, edges, 0, 3, 4) == -1
    # disconnected, and source equals target
    assert best_path_safety(3, [(0, 1, 9)], 0, 2) == -1
    assert best_path_safety(3, [], 1, 1) == float("inf")
    one = [(0, 1, 9)]
    assert best_path_safety_heap(3, one, 0, 2) == -1
```

FUNCTION AND TEST CASES

Sample call

```
solution(4, [(0,1,5),(1,3,4),(0,2,2),(2,3,8)], 0, 3, 6)
```

Test cases

Input	Expected
`(4, [(0,1,5),(1,3,4),(0,2,2),(2,3,8)], 0, 3, 6)`	`5`
`(3, [(0,1,7),(1,2,3)], 0, 2, 5)`	`-1`

STUDY PATH: question 52 of 167 (Core 90)

Priority score 2.92 | Dijkstra and shortest paths | asked 93 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 4 of 4

Minimum Direction Violations in a Directed Graph

ASKED 2026-07-19 | 93 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 10.6 | 30 DAYS 0 | PRIORITY 165/228

ABOUT THIS QUESTION

Topic: Core 90 - Dijkstra and shortest paths

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1466 Reorder Routes to Make All Paths Lead to the City Zero (related)

Frequency: Single report | Last asked: 2026-07-19 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/minimum-direction-violations-directed-graph

QUESTION

EXAMPLES

```
n = 5
edges = [[0, 1], [2, 1], [2, 3], [4, 3]]
start = 0
end = 4

path: 0 -> 1 -> 2 -> 3 -> 4
cost: 0 + 1 + 0 + 1 = 2
output: 2
```

APPROACH

Implement 0-1 BFS from memory and rehearse the deque invariant for zero-weight versus one-weight transitions.
Compare the proof and complexity with Dijkstra on the same transformed graph.
Test unreachable destinations, parallel edges, cycles, and start == end.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what a violation is (traversing an edge against its direction), whether edge costs are all 1, what to return when the end cannot be reached, and whether the graph has parallel edges or self loops.

BRUTE FORCE TO SAY OUT LOUD

Plain BFS ignoring direction minimizes edges, not violations, so say why it is wrong. Dijkstra with costs 0 and 1 works at O((n + m) log n).

OPTIMAL IDEA

Add every edge twice: forward with weight 0, backward with weight 1. Run 0-1 BFS with a deque: relax a weight-0 edge by pushing to the front, a weight-1 edge to the back. The deque always holds nodes whose distances differ by at most 1, so O(n + m).

DRY-RUN THESE CASES

Start equals end (0). A graph with no path. A path that needs every edge reversed. A shortcut that costs more edges but fewer violations.

LIKELY FOLLOW-UPS

General edge costs (Dijkstra), shortest path itself (parent pointers), or negative costs (Bellman-Ford).

PITFALLS

Using visited flags instead of a distance check. Pushing weight-0 edges to the back.

HOW TO THINK ABOUT IT

Going against an edge costs 1 and going with it costs 0, so it is a shortest-path problem with costs of only 0 and 1. Plain BFS counts edges, not violations, so it is the wrong tool. A deque lets 0-cost moves jump the queue (push front) and 1-cost moves wait their turn (push back), which is Dijkstra without the heap.

Basis: Related to LeetCode 1466, Reorder Routes: a shortest path where traveling an edge backwards costs 1.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, start, end):
    arcs = []
    for u, v in edges:
        arcs.append((u, v, 0))  # along the edge
        arcs.append((v, u, 1))  # against it
    dist = [INF] * n
    dist[start] = 0
    for _ in range(n - 1):
        for u, v, w in arcs:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    return dist[end]
```

TIME	SPACE
O(V * E)	O(V)

WHY IT IS SLOW

Every round scans all edges, V - 1 rounds: O(V * E). Costs of only 0 and 1 allow a deque search (0-1 BFS) in O(V + E).

OPTIMAL SOLUTION · 0-1 BFS on the doubled graph

TIME	SPACE
O(n + m)	O(n + m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is 0-1 BFS correct, and how does it compare with Dijkstra?

The deque always holds nodes whose distances differ by at most one: front elements are at distance d, back elements at d + 1. Relaxing a zero-weight edge keeps the distance at d (front), a one-weight edge makes d + 1 (back). That is the same order a heap would produce, so it is Dijkstra without the log factor: O(n + m) versus O((n + m) log n).

FOLLOW-UP 2 What if edge costs can be arbitrary non-negative numbers?

Use Dijkstra with a heap on the same doubled graph (forward cost = 0 or the edge cost, backward cost = the violation cost). With negative costs use Bellman-Ford and watch for negative cycles.

FOLLOW-UP 3 Edge costs are arbitrary non-negative numbers (different for each direction).

Use Dijkstra with a heap on the same doubled graph, giving each arc its own cost. 0-1 BFS only works because the costs are 0 or 1.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Minimum Direction Violations in a
# Directed Graph
#
# Examples: n = 5 edges = [[0, 1], [2, 1], [2, 3], [4, 3]]
# start = 0 end = 4 path: 0 -> 1 -> 2 -> 3 -> 4 cost: 0 +
# 1 + 0 + 1 = 2 output: 2
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is 0-1 BFS correct, and how does it compare with
# Dijkstra?
# 2. What if edge costs can be arbitrary non-negative
# numbers?
# 3. Edge costs are arbitrary non-negative numbers
# (different for each direction).
#
# INTERVIEW NOTES
# Going against an edge costs 1 and going with it costs 0,
# so it is a shortest-path problem with costs of only 0
# and 1. Plain BFS counts edges, not violations, so it is
# the wrong tool. A deque lets 0-cost moves jump the queue
# (push front) and 1-cost moves wait their turn (push
# back), which is Dijkstra without the heap.
#
# BRUTE FORCE (say it first): Bellman-Ford on the doubled
# graph
# Give every edge a forward copy with cost 0 and a
# backward copy with cost 1, and relax all edges V - 1
# times. Correct and simple, and the oracle for the 0-1
# BFS; it is slower because it repeats full passes over
# the edges. Time O(V * E). Space O(V).


import heapq
```

```
from collections import deque


# HELPERS for the brute force
INF = float("inf")


# BRUTE FORCE (say it first): Bellman-Ford on the doubled
# graph | time O(V * E)
def violations_bellman_ford(n, edges, start, end):
    INF = float("inf")
    dist = [INF] * n
    dist[start] = 0
    arcs = [(u, v, 0) for u, v in edges] + [
        (v, u, 1) for u, v in edges
    ]
    for _ in range(max(0, n - 1)):
        changed = False
        for u, v, w in arcs:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                changed = True
        if not changed:
            break
    return -1 if dist[end] == INF else dist[end]


# BASE QUESTION (optimal): 0-1 BFS on the doubled graph
# Idea: Add every edge twice: forward with weight 0,
# backward with weight 1. Shortest path with weights in
# {0, 1} is a deque BFS: push a zero-weight neighbour to
# the front and a one-weight neighbour to the back.
def min_violations(n, edges, start, end):
    # 0-1 BFS: following an edge is free, reversing costs
    # 1
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append((v, 0))
        graph[v].append((u, 1))
    dist = [INF] * n
    dist[start] = 0
    dq = deque([start])
    while dq:
        u = dq.popleft()
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                # cost 0 goes to the front, cost 1 to the
                # back
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    return -1 if dist[end] == INF else dist[end]


# FOLLOW-UP 3: Edge costs are arbitrary non-negative
```

```
# numbers (different for each direction).
# Answer: Use Dijkstra with a heap on the same doubled
# graph, giving each arc its own cost. 0-1 BFS only works
# because the costs are 0 or 1.
def cheapest_route(n, edges, start, end):
    # edges are (u, v, forward_cost, backward_cost)
    graph = [[] for _ in range(n)]
    for u, v, fwd, back in edges:
        graph[u].append((v, fwd))
        graph[v].append((u, back))
    dist = [INF] * n
    dist[start] = 0
    heap = [(0, start)]
    while heap:
```

```
        d, u = heapq.heappop(heap)
        # skip outdated heap entries
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return -1 if dist[end] == INF else dist[end]


if __name__ == "__main__":
    edges = [(0, 1), (2, 1), (2, 3), (4, 3)]
    # path 0 -> 1 -> 2 -> 3 -> 4 reverses two edges
    assert min_violations(5, edges, 0, 4) == 2
    assert min_violations(5, edges, 0, 0) == 0
    assert min_violations(3, [(0, 1)], 0, 2) == -1
    # the same graph with every reversal costing 1
    costed = [(u, v, 0, 1) for u, v in edges]
    assert cheapest_route(5, costed, 0, 4) == 2
    # different cost per direction
    assert cheapest_route(2, [(0, 1, 5, 2)], 0, 1) == 5
    assert cheapest_route(2, [(0, 1, 5, 2)], 1, 0) == 2
```

NOTES FROM CANDIDATES

Represent each original edge twice: the forward direction with weight 0 and the reverse direction with weight 1.
A regular BFS minimizes the number of edges, not the number of direction violations.
The linear-time target is O(n + m) time and space.

PREPARATION

Implement 0-1 BFS from memory and rehearse the deque invariant for zero-weight versus one-weight transitions.
Compare the proof and complexity with Dijkstra on the same transformed graph.
Test unreachable destinations, parallel edges, cycles, and start == end.